

دانشگاه علم و صنعت ایران

گزارش پروژه Polaris

اعضای گروه:

کوشا لاهوتی
سجاد میرجلیلی
مهدی معدنی پور

مقدمه

اپلیکیشن Polaris یک ابزار مدرن و پیشرفته اندرویدی برای مانیتورینگ جامع و تست عملکرد شبکه‌های موبایل (Drive Test) است. هدف اصلی این پروژه، ایجاد یک پلتفرم قابل اعتماد برای جمع‌آوری مداوم و دقیق پارامترهای شبکه سلولی در شرایط واقعی، اجرای تست‌های فعال شبکه (مانند پینگ و تست سرعت) و همگام‌سازی این داده‌ها با یک سرور مرکزی است. داده‌های جمع‌آوری شده می‌توانند برای تحلیل‌های آماری، شناسایی نقاط ضعف پوشش شبکه و بصری‌سازی عملکرد شبکه بر روی نقشه استفاده شوند. توسعه این برنامه با تمرکز کامل بر جدیدترین اصول مهندسی نرم‌افزار و Best Practices در اکوسیستم اندروید، مانند معماری لایه‌بندی شده، برنامه‌نویسی Reactive و Dependency Injection صورت گرفته تا محصول نهایی، برنامه‌ای پایدار، مقیاس‌پذیر، بهینه و قابل نگهداری باشد.

۱ ویژگی‌های اپلیکیشن

۱.۱ جمع‌آوری مداوم در پس‌زمینه:

تکنولوژی: Foreground Service

توضیحات: برنامه با اجرای یک سرویس پیش‌زمینه، تضمین می‌کند که فرآیند جمع‌آوری داده حتی در صورت بسته شدن رابط کاربری یا قفل بودن صفحه، توسط سیستم‌عامل متوقف نشود. این امر با نمایش یک نوتیفیکیشن دائمی به کاربر اطلاع داده می‌شود که نشان‌دهنده فعال بودن سرویس است.

۲.۱ ثبت پارامترهای دقیق شبکه:

تکنولوژی: Fused Location Provider, TelephonyManager

توضیحات: طیف وسیعی از پارامترهای شبکه سلولی با دقت بالا جمع‌آوری می‌شود:

(ا) کیفیت و قدرت سیگنال: مقادیر کلیدی مانند RSRP, RSRQ برای شبکه‌های LTE/5G

و RSCP برای WCDMA و RxLev برای GSM.

(ب) شناسه‌های سلول: شناسه‌های منحصر به فرد سلول شبکه مانند PLMN ID, Cell ID,

LAC, TAC که برای تحلیل‌های مکانی ضروری هستند.

(ج) فناوری شبکه: تشخیص و تفکیک دقیق بین نسل‌های مختلف شبکه از 2G تا 5G (شامل

حالت‌های NSA و SA).

(د) موقعیت مکانی: ثبت مختصات دقیق جغرافیایی (GPS) برای هر رکورد با استفاده از Fused

Location Provider گوگل که با ترکیب GPS, Wi-Fi و دکل‌های مخابراتی، دقیق‌ترین

و بهینه‌ترین موقعیت را ارائه می‌دهد.

۳.۱ ذخیره‌سازی محلی داده‌ها:

تکنولوژی: Room Database

توضیحات: تمام متریک‌ها و نتایج تست‌ها به صورت محلی در یک دیتابیس Room ذخیره می‌شوند. این کار باعث می‌شود داده‌ها در صورت عدم دسترسی به اینترنت یا خاموش شدن دستگاه از بین نروند.

۴.۱ اجرای تست‌های شبکه:

تکنولوژی: Foreground Service, WorkManager

توضیحات: برنامه از سه نوع تست پشتیبانی می‌کند:

- (ا) تست‌های دستی: کاربر به صورت مستقیم از طریق UI اجرا می‌کند.
- (ب) تست‌های زمان‌بندی شده (**Scheduled**): تست‌هایی که برای اجرا در یک زمان مشخص در آینده تنظیم شده‌اند و توسط WorkManager به صورت تضمینی اجرا می‌شوند.
- (ج) تست‌های دوره‌ای (**Periodic**): تست‌هایی که در فواصل زمانی معین (مثلاً هر ۵ دقیقه) تکرار می‌شوند و منطق اجرای آن‌ها برای دقت بالا در Foreground Service پیاده‌سازی شده است.

۵.۱ همگام‌سازی با سرور (**Synchronization**):

تکنولوژی: WorkManager, OkHttp, Retrofit

توضیحات: ارتباط با سرور به صورت هوشمند مدیریت می‌شود:

- (ا) همگام‌سازی متریک‌ها و نتایج تست: یک WorkManager دوره‌ای وظیفه ارسال داده‌های همگام‌نشده به سرور را بر عهده دارد و تنها در صورت اتصال به اینترنت اجرا می‌شود.
- (ب) همگام‌سازی تنظیمات تست: برنامه به صورت دوره‌ای لیست تست‌های جدید را از سرور دریافت کرده و تست‌هایی که از روی سرور حذف شده‌اند را به صورت محلی نیز پاک می‌کند.

۶.۱ رابط کاربری مدرن (**Modern UI/UX**):

تکنولوژی: Jetpack Compose

توضیحات: کل رابط کاربری با استفاده از Jetpack Compose، ابزار مدرن و اعلانی گوگل، ساخته شده است. این امر منجر به کدی خواناتر، توسعه سریع‌تر و عملکرد بهتر UI شده است. ناوبری بین صفحات با استفاده از Bottom Navigation Bar و Navigation Compose پیاده‌سازی شده است.

۷.۱ تنظیمات قابل شخصی سازی:

تکنولوژی: Jetpack DataStore

توضیحات: کاربران می توانند تنظیمات مختلفی مانند تم برنامه (روشن، تاریک، پیش فرض سیستم)، فاصله زمانی جمع آوری داده و فعال/غیرفعال کردن همگام سازی خودکار را تغییر دهند. این تنظیمات به صورت پایدار با استفاده از DataStore ذخیره می شوند.

۲ معماری برنامه

این اپلیکیشن بر اساس معماری توصیه شده توسط گوگل برای توسعه اندروید مدرن، با تاکید بر رابط کاربری واکنشی (Reactive UI) و تفکیک مسئولیت ها (Separation of Concerns) ساخته شده است.

۱.۲ لایه رابط کاربری (UI Layer):

تکنولوژی: Jetpack Compose

شرح: این لایه مسئول نمایش اطلاعات به کاربر و دریافت ورودی های اوست. کامپوننت های این لایه (Composable Functions) به صورت Stateless طراحی شده اند، به این معنی که وضعیت (State) خود را مستقیماً مدیریت نمی کنند، بلکه آن را از ViewModel دریافت کرده و رویدادها (Events) را به آن ارسال می کنند. این الگو تست پذیری و قابلیت استفاده مجدد کامپوننت ها را به شدت افزایش می دهد.

۲.۲ لایه ViewModel:

تکنولوژی: AndroidX ViewModel

شرح: این لایه به عنوان پلی بین UI و لایه داده عمل می کند. ViewModel ها وضعیت UI را در قالب StateFlow نگهداری می کنند و منطق مربوط به تعاملات کاربر را مدیریت می نمایند. آن ها از تغییرات پیکربندی (مانند چرخش صفحه) جان سالم به در می برند و از بارگذاری مجدد داده ها جلوگیری می کنند.

۳.۲ لایه Repository:

تکنولوژی: الگوی طراحی Repository

شرح: این لایه منابع داده را از بقیه برنامه جدا (Abstract) می کند. ViewModel ها فقط با Repository در ارتباط هستند و هیچ اطلاعی از نحوه ذخیره یا دریافت داده ها (از دیتابیس محلی یا سرور) ندارند. این الگو امکان تغییر منبع داده را بدون تغییر در منطق برنامه فراهم کرده و تست پذیری را آسان می کند.

۴.۲ لایه داده (Data Layer):

محلی (Local): (ا) Room: برای ذخیره‌سازی داده‌های ساختاریافته (متریک‌ها و تست‌ها). Room با ارائه بررسی صحت کوئری‌ها در زمان کامپایل، از بروز خطاهای رایج SQLite جلوگیری می‌کند.
(ب) DataStore: برای ذخیره‌سازی داده‌های ساده کلید-مقدار (تنظیمات کاربر) به صورت ناهمگام و ایمن.

ریموت (Remote): (ا) Retrofit: برای ساخت کلاینت HTTP و تعریف API‌ها به صورت خوانا و ایمن.
(ب) Kotlinx Serialization: برای تبدیل (Serialize/Deserialize) خودکار آبجکت‌های Kotlin به JSON و بالعکس.

۵.۲ تزریق وابستگی (Dependency Injection):

تکنولوژی: Hilt

شرح: Hilt فرآیند ایجاد و ارائه وابستگی‌ها (مانند Repository یا ApiService) به کلاس‌های دیگر را خودکار می‌کند. این کار باعث کاهش چشمگیر کدهای تکراری (Boilerplate) و جداسازی کامل کامپوننت‌ها از یکدیگر می‌شود.

۶.۲ عملیات ناهمگام (Asynchronous Operations):

تکنولوژی: Flow و Kotlin Coroutines

شرح: تمام عملیات زمان‌بر مانند کار با دیتابیس، درخواست‌های شبکه و اجرای تست‌ها در Coroutineها انجام می‌شوند تا از مسدود شدن رشته اصلی (Main Thread) و هنگ کردن برنامه جلوگیری شود. Flow برای مدیریت جریان‌های داده‌ای که در طول زمان تغییر می‌کنند (مانند لیست متریک‌ها از دیتابیس) استفاده می‌شود.

۳ چالش‌ها و راه‌حل‌ها

۱.۳ چالش: جمع‌آوری پایدار داده در پس‌زمینه

مشکل: سیستم‌عامل اندروید برای بهینه‌سازی مصرف باتری، فرآیندهای پس‌زمینه را به شدت محدود می‌کند.

راه‌حل: استفاده از یک Foreground Service. این نوع سرویس با نمایش یک نوتیفیکیشن دائمی، به کاربر و سیستم‌عامل اعلام می‌کند که در حال انجام یک کار مهم و مداوم است. این کار بالاترین اولویت را برای فرآیند برنامه تضمین می‌کند و از توقف آن توسط سیستم جلوگیری می‌کند.

۲.۳ چالش: مدیریت وضعیت پیچیده UI به صورت واکنشی

مشکل: UI باید به تغییرات همزمان از منابع مختلف (وضعیت سرویس، تایمر، لیست متریک‌ها از دیتابیس، وضعیت همگام‌سازی) واکنش نشان دهد.

راه‌حل: استفاده از یک StateFlow واحد در ViewModel که با استفاده از عملگر combine، چندین Flow ورودی را ترکیب کرده و یک آبجکت UiState جامع تولید می‌کند. هر تغییری در هر یک از منابع ورودی، باعث تولید یک UiState جدید و به‌روزرسانی خودکار رابط کاربری می‌شود.

۳.۳ چالش: اجرای تست‌های دوره‌ای با فواصل زمانی کوتاه (کمتر از ۱۵ دقیقه)

مشکل: PeriodicWorkRequest در WorkManager دارای حداقل فاصله زمانی ۱۵ دقیقه است که برای تست‌های با فرکانس بالا (مثلاً هر ۵ دقیقه) مناسب نیست.

راه‌حل: پیاده‌سازی یک رویکرد ترکیبی:

(ا) تست‌های دوره‌ای (**Periodic**): منطق اجرای این تست‌ها در NetworkMetricService پیاده‌سازی شده است. از آنجایی که این سرویس همیشه در حال اجراست، می‌تواند یک تایمر داخلی را با هر فاصله‌ی زمانی دلخواه مدیریت کند.

(ب) تست‌های زمان‌بندی شده (**Scheduled**) و همگام‌سازی‌ها: این وظایف همچنان توسط WorkManager مدیریت می‌شوند، زیرا WorkManager برای کارهایی که باید به صورت تضمینی (حتی پس از راه‌اندازی مجدد دستگاه) و در زمان بهینه اجرا شوند، بهترین گزینه است.

۴.۳ چالش: مدیریت مجوزهای حساس (Permissions)

مشکل: برنامه به مجوزهای حساسی مانند READ_PHONE_STATE, ACCESS_FINE_LOCATION و POST_NOTIFICATIONS نیاز دارد.

راه‌حل: ایجاد یک کامپوننت اختصاصی به نام HandlePermissions. این کامپوننت در ابتدای برنامه، قبل از نمایش هرگونه رابط کاربری اصلی، تمام مجوزهای لازم را از کاربر درخواست می‌کند. این الگو تضمین می‌کند که منطق اصلی برنامه تنها زمانی اجرا می‌شود که تمام پیش‌نیازهای آن فراهم شده باشد و تجربه کاربری بهتری را رقم می‌زند.

۴ توضیح فایل‌های کلیدی

در این بخش، فایل‌ها و کامپوننت‌های مهم برنامه با جزئیات بیشتری شرح داده می‌شوند.

۱.۴ MainActivity.kt

نقش: تنها Activity و نقطه ورود اصلی برنامه. این فایل به عنوان میزبان اصلی برای تمام کامپوننت‌های Jetpack Compose عمل می‌کند.

توابع کلیدی: (ا) onCreate(): در این متد، مجوزهای لازم تعریف شده و setContent فراخوانی می‌شود. کامپوننت HandlePermissions در اینجا استفاده می‌شود تا اطمینان حاصل شود که برنامه تنها پس از دریافت مجوزهای لازم، UI اصلی را نمایش می‌دهد.

(ب) MainAppScaffold(): این کامپوننت Scaffold اصلی برنامه را تعریف می‌کند که شامل ساختار کلی صفحات مانند TopAppBar و BottomNavigationBar است. همچنین NavHost برای مدیریت ناوبری بین صفحات در اینجا قرار دارد.

جایگاه در معماری: لایه UI. این فایل نقطه شروع و ریشه درخت کامپوننت‌های Compose است.

۲.۴ service/NetworkMetricService.kt

نقش: قلب تپنده جمع‌آوری داده. این Foreground Service دو وظیفه اصلی و موازی دارد:

(ا) اجرای یک حلقه دائمی برای جمع‌آوری اطلاعات موقعیت مکانی و پارامترهای شبکه سلولی و ذخیره آن‌ها در دیتابیس Room.

(ب) اجرای یک حلقه دیگر برای بررسی و اجرای تست‌های دوره‌ای (Periodic) که توسط کاربر تنظیم شده‌اند.

توابع کلیدی: (ا) onStartCommand(): برای مدیریت شروع و توقف سرویس از طریق Intentها.

(ب) startMetricCollection(): حلقه‌های جمع‌آوری داده و اجرای تست‌های دوره‌ای را آغاز می‌کند.

(ج) gatherAndStoreCellularMetricData(): منطق اصلی برای دسترسی به TelephonyManager و LocationManager و ساخت آبجکت NetworkMetric.

جایگاه در معماری: یک کامپوننت اصلی اندروید که در لایه UI قرار می‌گیرد (زیرا با چرخه حیات سیستم‌عامل در ارتباط است) اما منطق آن مستقیماً با لایه داده (Repository) در تعامل است.

۳.۴ service/TestExecutor.kt

نقش: موتور اجرایی تست‌ها. این کلاس مسئولیت فنی اجرای هر نوع تست را بر عهده دارد. این جداسازی باعث می‌شود منطق اجرای تست‌ها تمیز و قابل تست باشد.

توابع کلیدی: (ا) execute(test: Test): متد عمومی که یک Test را دریافت کرده و بر اساس نوع آن، متد خصوصی مربوطه را فراخوانی می‌کند.

(ب) `executePingTest()`, `executeDnsTest()` و...: متدهای خصوصی که هر کدام منطق یک نوع تست خاص را پیاده‌سازی می‌کنند (مثلاً اجرای دستور `ping` یا برقراری اتصال `HTTP`).

جایگاه در معماری: یک کلاس کمکی (`Helper/Utility`) که در لایه `Domain` (منطق کسب‌وکار) قرار می‌گیرد، زیرا منطق اصلی نحوه اجرای یک تست را تعریف می‌کند.

۴.۴ `service/TestSchedulerWorker.kt`

نقش: یک `WorkManager Worker` که به صورت دوره‌ای (هر ۱۵ دقیقه) توسط سیستم‌عامل اجرا می‌شود. وظیفه آن اجرای تست‌های زمان‌بندی شده‌ی یک‌باره (`Scheduled`) و همچنین همگام‌سازی لیست تست‌ها با سرور است.

توابع کلیدی: `doWork()` (۱) این متد توسط `WorkManager` فراخوانی می‌شود و شامل منطق دریافت تست‌های موعده رسیده از `Repository` و اجرای آن‌ها از طریق `TestExecutor` است.

جایگاه در معماری: مانند سرویس، یک کامپوننت اندروید در لایه `UI` است که برای اجرای منطق در پس‌زمینه با لایه داده تعامل دارد.

۵.۴ `ui/viewmodel/` (شامل `MainViewModel.kt` و `TestsViewModel.kt`)

نقش: این پکیج شامل `ViewModel` های برنامه است که وضعیت `UI` را مدیریت می‌کنند.

توابع کلیدی: `uiState: StateFlow` (۱) هر `ViewModel` یک `StateFlow` عمومی دارد که وضعیت کامل صفحه مربوطه را در قالب یک `data class` نگهداری می‌کند.

(ب) توابع رویداد (`Event Handlers`) مانند `onToggleCollection()` یا `onSyncClicked()`: این توابع از `UI` فراخوانی شده و منطق مربوطه (مانند شروع سرویس یا فراخوانی `Repository`) را اجرا می‌کنند.

جایگاه در معماری: لایه `ViewModel`.

۶.۴ `data/local/AppDatabase.kt` و `DAO` ها

نقش: تعریف کامل دیتابیس `Room` برنامه. `AppDatabase` کلاس اصلی است که تمام جداول (`Entities`) و رابط‌های دسترسی به داده (`DAOs`) را تعریف می‌کند. هر `DAO` (مانند `NetworkMetricDao` و `TestDao`) شامل کوئری‌های `SQL` برای تعامل با جدول مربوطه است.

جایگاه در معماری: بخش اصلی لایه داده محلی (`Local Data Layer`).

۷.۴ data/repository/ (شامل OfflineFirstTestsRepository.kt و...)

نقش: این پکیج الگوی Repository را پیاده‌سازی می‌کند. کلاس‌های Repository تنها نقطه دسترسی به داده برای ViewModel‌ها هستند. آن‌ها منطق تصمیم‌گیری در مورد اینکه داده از دیتابیس محلی خوانده شود یا از سرور را پنهان می‌کنند.

توابع کلیدی: (ا) syncTests(), syncMetrics(): توابعی که با ApiService برای همگام‌سازی داده‌ها با سرور ارتباط برقرار می‌کنند.

(ب) getManualTests(), getAllMetricsFlow(): توابعی که داده‌ها را از DAOهای محلی برای نمایش در UI می‌خوانند.

جایگاه در معماری: لایه Repository که بین لایه ViewModel و لایه داده قرار می‌گیرد.

این پروژه بر اساس Clean Architecture طراحی شده؛ معماری‌ای که هدفش جداسازی کامل لایه‌ها و وابستگی‌هاست تا کد قابل نگهداری، تست‌پذیر و توسعه‌پذیر باشد.

لایه‌های اصلی در این پروژه:

۱. Domain

- (ا) شامل Value Object, Entities و قوانین اصلی کسب‌وکار.
- (ب) هیچ وابستگی به تکنولوژی یا فریم‌ورک ندارد.
- (ج) مثال: تعریف کلاس User یا Group فقط با قوانین دامین.

۲. Application

- (ا) پیاده‌سازی Use Case ها (مثل مدیریت کاربر، ارسال پیام).
- (ب) فقط با اینترفیس‌ها کار می‌کند، نه پیاده‌سازی‌ها.
- (ج) از CQRS و MediatR (در بیشتر پروژه‌ها) برای تفکیک Command/Query استفاده می‌شود.

۳. Infrastructure

- (ا) پیاده‌سازی جزئیات تکنیکی مثل دیتابیس (EF Core), Repositories, سرویس‌های خارجی (Auth, Storage,...).
- (ب) وابسته به Application است، چون اینجا قراردادهای تعریف شده در Application پیاده‌سازی می‌شوند.

۴. WebAPI

- (ا) نقطه ورود برای کلاینت‌ها.
- (ب) شامل Controller ها، Middleware ها، Endpoint های HTTP.
- (ج) (Dependency Injection) DI اینجا همه لایه‌ها را به هم متصل می‌کند.

Domain

Entities: شامل کلاس‌های موجودیت (User, Group و ...)، یعنی مدل‌های اصلی دامین و قوانین درونی آن‌ها.

Application

Common: شامل helper ها، Behavior ها مثل Pipeline برای MediatR و ابزارهای عمومی سطح اپلیکیشن.

Features: هر یوزکیس/سرویس خاص اینجا پیاده شده مثلاً Authentication, Messaging, GroupManagement معمولاً به صورت پوشه به‌ازای هر فیچر.

Interfaces: تعریف اینترفیس‌هایی که زیرساخت باید پیاده کند مثل IUserRepository, IMessageService

Infrastructure

Data: پیاده‌سازی DbContext با EF Core، کانفیگ جداول و ارتباطات.

Repositories: پیاده‌سازی اینترفیس‌های Repositories تعریف‌شده در Application

Services: پیاده‌سازی سرویس‌های خارجی Auth، ایمیل، فایل‌استوریج و ... یا هر چیز وابسته

WebAPI

Controllers: شامل کنترلرهای REST مثل UserController، AuthController، MessageController

Program.cs: نقطه ورود اپلیکیشن ساخت WebApplication، رجیستر کردن DI، Middlewareها

appsettings.json: تنظیمات محیط ConnectionString دیتابیس، JWT، Logging

Domain

۸.۴ .۱ User

این انتیتی، موجودیت کاربر سیستم را تعریف می‌کند. هدف اصلی آن مدیریت هویت و فرآیندهای احراز هویت (Authentication) است. هر رکورد در جدول متناظر با این انتیتی، یک کاربر منحصر به فرد را نمایندگی می‌کند. فیلد Id یک شناسه یکتا برای هر کاربر است. فیلد Username نام کاربری است که کاربر با آن وارد سیستم می‌شود. فیلدهای PasswordHash و PasswordSalt برای ذخیره‌سازی امن رمز عبور استفاده می‌شوند. به جای ذخیره مستقیم رمز عبور، یک هش رمزنگاری شده از آن به همراه یک مقدار تصادفی (Salt) ذخیره می‌شود تا امنیت در صورت نشت اطلاعات پایگاه داده، به شدت افزایش یابد. فیلد NetworkInformationId یک کلید خارجی است که به انتیتی NetworkMeasurement اشاره دارد و می‌تواند آخرین وضعیت اطلاعات شبکه کاربر را در خود نگهداری کند. این ارتباط اختیاری است و یک کاربر ممکن است اطلاعات شبکه ثبت‌شده‌ای نداشته باشد.

۹.۴ .۲ Test

این انتیتی یک تست شبکه را تعریف می‌کند که قرار است اجرا شود. در واقع، این موجودیت یک الگو یا پیکربندی برای یک وظیفه (Task) است که می‌تواند یک یا چند بار اجرا شود. برای مثال، می‌توان یک تست "پینگ به سرور گوگل" یا "تست سرعت دانلود" تعریف کرد. فیلد Id شناسه یکتای هر تعریف تست است. Name و Type به ترتیب نام و نوع تست را مشخص می‌کنند. ParametersJson یک رشته با فرمت JSON است که پارامترهای مورد نیاز برای اجرای تست را در خود ذخیره می‌کند. این روش انعطاف‌پذیری بالایی ایجاد می‌کند، زیرا می‌توان پارامترهای مختلفی را بدون تغییر ساختار دیتابیس برای انواع تست‌ها ذخیره کرد. IsEnabled مشخص می‌کند که آیا این تست فعال است و باید اجرا شود یا خیر. فیلدهای ScheduledTimestamp و IntervalSeconds برای زمان‌بندی اجرای تست‌ها به کار می‌روند. ScheduledTimestamp برای اجرای تست در یک زمان مشخص و IntervalSeconds برای اجرای تکراری تست در بازه‌های زمانی معین استفاده

می‌شود. IsCompleted وضعیتی را نشان می‌دهد که آیا یک تست زمان‌بندی شده (یک‌باره) انجام شده است یا خیر.

۱۰.۴ ۳. TestResult

این انتیتی نتیجه‌ی حاصل از اجرای یک تست را ذخیره می‌کند. هر بار که یک Test تعریف شده اجرا می‌شود، یک رکورد جدید از نوع TestResult ایجاد می‌گردد تا خروجی و جزئیات آن اجرا را ثبت کند. Id شناسه یکتای هر نتیجه تست است. Timestamp زمان دقیق اجرای تست را به صورت یک عدد نگهداری می‌کند. TestType و TargetHost به ترتیب نوع تست انجام شده مثلاً ("Ping") و میزبان هدف (مثلاً "8.8.8.8") را مشخص می‌کنند. ResultValue مقدار اصلی نتیجه تست را به صورت رشته ذخیره می‌کند (مثلاً "50ms" یا "85Mbps"). IsSuccess یک مقدار بولین (boolean) است که موفقیت‌آمیز بودن یا نبودن اجرای تست را نشان می‌دهد. Details می‌تواند شامل اطلاعات بیشتر، جزئیات خطا یا خروجی کامل‌تر تست باشد. Filed TestId یک کلید خارجی است که این نتیجه را به Test اصلی که آن را ایجاد کرده، متصل می‌کند. این ارتباط به ما اجازه می‌دهد تا تمام نتایج مربوط به یک تست خاص را پیدا کنیم.

۱۱.۴ ۴. NetworkMeasurement

این انتیتی برای ثبت و ذخیره‌سازی مجموعه‌ای از پارامترهای فنی و دقیق شبکه در یک لحظه زمانی خاص طراحی شده است. این اطلاعات معمولاً مربوط به شبکه‌های موبایل (مانند 3G, 4G/LTE) و موقعیت جغرافیایی کاربر در هنگام ثبت اطلاعات است. TimeStamp زمان ثبت این اندازه‌گیری را نشان می‌دهد. Longitude و Latitude موقعیت جغرافیایی دستگاه را مشخص می‌کنند. NetworkType نوع شبکه (مثلاً "LTE", "WIFI", "HSPA") را تعیین می‌کند. سایر فیلدها مانند PLMNId, Tac, CellId, RSRQ, RSRP و... پارامترهای بسیار تخصصی شبکه سلولی هستند که به ترتیب شناسه اپراتور، کد منطقه ردیابی، شناسه دکل مخابراتی، و کیفیت و قدرت سیگنال را اندازه‌گیری می‌کنند. بسیاری از این فیلدها اختیاری (nullable) هستند، زیرا فقط برای نوع خاصی از شبکه کاربرد دارند (مثلاً RSRP فقط برای شبکه LTE معنا دارد).

۱۲.۴ ۵. DeletedTest

این انتیتی یک موجودیت ساده برای ردیابی تست‌هایی است که از سیستم حذف شده‌اند. هدف اصلی آن معمولاً همگام‌سازی (Synchronization) داده‌ها بین سرور و کلاینت‌ها است. وقتی یک Test از جدول اصلی حذف می‌شود، شناسه‌ی آن در این جدول ثبت می‌گردد. به این ترتیب، اپلیکیشن‌های کلاینت می‌توانند با بررسی این جدول، متوجه شوند که کدام تست‌ها را باید از حافظه محلی خود حذف کنند. Id شناسه یکتای خود رکورد حذف است. DeletedTestId شناسه (Id) تستی که از جدول Test حذف شده است را در خود نگهداری می‌کند.

Application

۱۳.۴ NetworkMeasurements

این بخش به عنوان مرکز جمع‌آوری و مدیریت داده‌های خام عملکرد شبکه عمل می‌کند و نقشی حیاتی در تحلیل کیفیت سرویس دارد. کلاس `SaveNetworkMeasurementCommand` یک دستور قدرتمند برای ثبت اطلاعات لحظه‌ای شبکه است که معمولاً از دستگاه‌های موبایل کاربران ارسال می‌شود. این دستور حاوی جزئیات فنی دقیقی مانند نوع شبکه، قدرت سیگنال، و مشخصات دکل مخابراتی متصل به آن است. کنترل‌کننده (Handler) مربوط به این دستور، داده‌ها را اعتبارسنجی کرده و با استفاده از `INetworkMeasurementRepository` آن‌ها را در پایگاه داده ذخیره می‌کند. از سوی دیگر، `GetNetworkMeasurementCommand` برای بازیابی این اطلاعات به کار می‌رود. برای مثال، می‌توان از آن برای نمایش تاریخچه وضعیت شبکه یک کاربر روی نقشه یا تحلیل مناطق با پوشش دهی ضعیف استفاده کرد و به این ترتیب به درک عمیق‌تری از تجربه کاربری رسید.

۱۴.۴ TestResults فیچر

این ماژول به صورت تخصصی برای ثبت و نگهداری خروجی‌های حاصل از اجرای تست‌های شبکه طراحی شده و به نوعی دفتر گزارش عملکرد سیستم است. هر بار که یک تست تعریف شده در سیستم اجرا می‌شود، یک نمونه از `SaveTestResultCommand` ایجاد شده و به سمت سرور ارسال می‌گردد. این دستور تمام جزئیات کلیدی نتیجه، از جمله مقدار به دست آمده، موفقیت یا شکست تست، و هرگونه پیام خطا را در خود جای می‌دهد. سپس این اطلاعات برای همیشه در پایگاه داده ثبت می‌شوند. دستور `GetTestResultsCommand` امکان دسترسی به این تاریخچه غنی را فراهم می‌سازد. توسعه‌دهندگان می‌توانند با استفاده از آن، گزارش‌های تحلیلی پیچیده‌ای تولید کنند؛ برای مثال، میانگین زمان پینگ به یک سرور خاص در یک ماه گذشته را محاسبه کرده و نمودارهای روند عملکرد را ترسیم نمایند.

۱۵.۴ Tests فیچر

این قسمت مغز متفکر سیستم برای تعریف، زمان‌بندی و مدیریت تست‌های شبکه است و مجموعه‌ای جامع از عملیات‌ها را پوشش می‌دهد. دستور `SaveTestCommand` به مدیران سیستم اجازه می‌دهد تا تست‌های جدیدی با پارامترهای دلخواه تعریف کنند، مثلاً یک تست سرعت دانلود که هر ساعت یک بار به طور خودکار اجرا شود. این تنظیمات سپس در پایگاه داده ذخیره می‌شوند. مکانیزم همگام‌سازی کلاینت‌ها با سرور از طریق دستورهای `GetLatestTestsCommand` و `GetDeletedTestsCommand` بسیار هوشمندانه طراحی شده است؛ کلاینت‌ها به جای دریافت کل لیست تست‌ها در هر بار، فقط تغییرات جدید و تست‌های حذف شده را دریافت می‌کنند که این امر کارایی را به شدت افزایش می‌دهد. دستور `DeleteTestCommand` نیز به جای حذف فیزیکی، یک تست را به صورت منطقی حذف می‌کند و شناسه آن را در جدول دیگری ثبت می‌کند تا کلاینت‌های آفلاین نیز پس از اتصال از این تغییر مطلع شوند.

۱۶.۴ فیچر Users

این بخش به طور کامل بر مدیریت چرخه حیات کاربران و تامین امنیت دسترسی آن‌ها متمرکز است. دستور `SignUpCommand` فرآیند ثبت‌نام کاربر جدید را مدیریت می‌کند. کنترل‌کننده این دستور پس از دریافت اطلاعات، وظیفه‌ای حیاتی بر عهده دارد: تبدیل رمز عبور کاربر به یک مقدار هش شده امن با استفاده از یک کلید تصادفی (Salt) قبل از ذخیره‌سازی. این کار امنیت حساب‌های کاربری را تضمین می‌کند. در طرف دیگر، `LoginCommand` فرآیند ورود را کنترل می‌کند. پس از اینکه کاربر نام کاربری و رمز عبور خود را ارسال کرد، سیستم رمز وارد شده را با مقدار هش ذخیره شده مقایسه می‌کند و در صورت تطابق، یک توکن JWT تولید کرده و برای کاربر ارسال می‌کند. این توکن از آن پس به عنوان کلید دسترسی کاربر برای تمام درخواست‌های بعدی به سرور عمل می‌کند.

۱۷.۴ Repositories

این پوشه معماری نرم‌افزار و پایبندی آن به اصول طراحی تمیز (Clean Architecture) را به نمایش می‌گذارد. اینترفیس‌ها مانند `IUserRepository` یا `ITestRepository` به عنوان یک قرارداد عمل می‌کنند که مشخص می‌کند چه عملیات‌هایی بر روی داده‌ها قابل انجام است، بدون اینکه منطق برنامه را به نحوه پیاده‌سازی این عملیات‌ها وابسته کنند. این انتزاع، انعطاف‌پذیری فوق‌العاده‌ای به پروژه می‌بخشد و اجازه می‌دهد در آینده تکنولوژی پایگاه داده را بدون تغییر در منطق اصلی برنامه، تعویض کنید. اینترفیس `IUnitOfWork` نیز نقش یک هماهنگ‌کننده برای تراکنش‌ها را ایفا می‌کند. این الگو تضمین می‌کند که یک سری از عملیات‌های مرتبط با پایگاه داده، مانند ثبت‌نام کاربر و ایجاد اولین تست برای او، یا به طور کامل با موفقیت انجام شوند یا در صورت بروز خطا، همگی با هم لغو گردند تا پایگاه داده در وضعیت ناسازگار قرار نگیرد.

Infrastructure

۱۸.۴ بخش Data:

این بخش مسئول مستقیم ارتباط با پایگاه داده است. کلاس `ApplicationDbContext` به عنوان نماینده پایگاه داده در کد عمل می‌کند و تمام جداول و روابط بین آن‌ها را مدیریت می‌کند. پوشه `Migrations` نیز تاریخچه‌ای از تمام تغییرات ساختاری پایگاه داده را نگهداری می‌کند تا بتوان نسخه پایگاه داده را به راحتی به‌روزرسانی یا بازگردانی کرد.

۱۹.۴ بخش Repositories:

این پوشه شامل پیاده‌سازی‌های واقعی و عینی اینترفیس‌هایی است که قبلاً تعریف شده است. برای مثال کلاس `TestRepository` حاوی کدهای واقعی برای اضافه کردن، خواندن یا حذف یک تست از پایگاه داده با استفاده از `Entity Framework Core` است. کلاس `UnitOfWork` نیز وظیفه دارد تا تمام تغییرات انجام شده در یک درخواست را به صورت یکجا و به عنوان یک تراکنش واحد در پایگاه داده ذخیره کند.

۲۰.۴ بخش Services:

این قسمت شامل سرویس‌های فنی است که به طور مستقیم با منطق اصلی کسب‌وکار در ارتباط نیستند اما برای عملکرد برنامه ضروری هستند. کلاس `HashService` وظیفه امنیتی هش کردن و بررسی صحت رمزهای عبور را بر عهده دارد. کلاس `JwtService` نیز برای ساختن و اعتبارسنجی توکن‌های امنیتی ورود کاربران به کار می‌رود تا هویت آن‌ها در هر درخواست تأیید شود.

WebApi

WebAPI بیرونی‌ترین و بالاترین لایه اپلیکیشن محسوب می‌شود که به عنوان دروازه ورودی اصلی برای تمام درخواست‌های خارجی عمل می‌کند. این لایه مسئولیت دریافت درخواست‌های HTTP از سمت کلاینت‌ها مانند اپلیکیشن موبایل یا وب را بر عهده دارد و آن‌ها را به لایه‌های داخلی‌تر سیستم هدایت می‌کند. مهم‌ترین بخش این لایه، پوشه `Controllers` است که به ازای هر فیچر اصلی یک کنترلر جداگانه دارد؛ برای مثال `UserController` تمام درخواست‌های مربوط به کاربران مانند ثبت‌نام و ورود را مدیریت می‌کند. این کنترلرها خودشان منطق اصلی را پیاده‌سازی نمی‌کنند، بلکه مانند یک واسطه عمل کرده و پس از دریافت و اعتبارسنجی اولیه یک درخواست، آن را به یک دستور یا کوئری مشخص در لایه اپلیکیشن ارسال می‌کنند. پس از دریافت پاسخ از لایه‌های داخلی، کنترلر وظیفه دارد آن نتیجه را در قالب یک پاسخ استاندارد HTTP با فرمت JSON بسته‌بندی کرده و به کلاینت بازگرداند. فایل `Program.cs` نیز در این لایه، نقطه شروع برنامه است که تمام سرویس‌ها، وابستگی‌ها و تنظیمات کلی اپلیکیشن در آن پیکربندی و راه‌اندازی می‌شوند.